Clase 4 – Construcción y verificación de un pipeline

Ejecución, trazabilidad y tratamiento de errores

Prof. Mag. Ing. Mat. Diego Fernández
Diploma en Ciencia de Datos e Inteligencia Artificial Aplicada

Modalidad: encuentro virtual práctico de 90 minutos. Caso acumulativo: Comercial Horizonte, con unidades monetarias (UM) ficticias. Fechas del CSV y de ejecución simuladas; no son el calendario del curso. La Semana 2 incluye además **2 horas de trabajo autónomo total**, compartidas por las Clases 3 y 4.

La pregunta de hoy

¿Podemos volver a ejecutar el mismo procedimiento y explicar de dónde salió cada resultado? El agente escribe el código; nosotros revisamos las decisiones, los archivos y las pruebas. No se evalúa escribir Python de memoria.

1. Qué aprenderemos y por qué pertenece al Módulo 6

Al terminar, podremos seguir el recorrido **leer, validar, calcular y guardar**; comprobar resultados con P1–P8; y conservar una ejecución que otra persona pueda revisar. Esto introduce trabajo propio de **ingeniería de datos**: transformar entradas en productos con reglas y controles. También prepara prácticas necesarias para MLOps: versiones, pruebas, repetición y registro de fallos. Esta semana no entrena ni despliega un modelo de aprendizaje automático.

Acuerdo de trabajo: objetivo → archivos autorizados → plan → ejecución con herramientas → pruebas y evidencias → revisión humana → nueva versión cuando corresponde. Original intacto. No publicar, enviar mensajes ni usar APIs, credenciales o carpetas ajenas.

Cinco palabras útiles. *Script*: archivo de instrucciones. *Pipeline*: secuencia que convierte entradas en productos. *Trazabilidad*: poder volver del resultado a la fuente y a la regla utilizada. *Fixture*: archivo pequeño creado para una prueba. *Regresión*: repetir pruebas anteriores después de corregir el programa.

2. Agenda posible en 90 minutos

Minutos	Trabajo y evidencia principal
0–8	Preparar entradas y predecir qué debe ocurrir.
8–23	Generar el script y leer sus decisiones principales.
23–38	Ejecutar el caso base y revisar P1–P6 de forma agrupada.
38–53	P7: cambio de política, igualdad y redondeo.
53–63	P8: conjunto sin filas válidas; demostración docente de detención D1.
63–73	R1: repetir la misma versión y comparar resultados.
73–83	Corregir un fallo concreto y repetir su prueba.
83–90	Revisar los tres productos, registrar pendientes y cerrar E2.

Organización: el docente conduce una ejecución común y los estudiantes la siguen desde sus computadoras. P1–P6 comparten evidencia del caso base; no requieren seis ejecuciones iguales. Si una co-

rrección exige más tiempo, la regresión completa pasa al trabajo autónomo. Una prueba pendiente sigue pendiente: no se aprueba por falta de tiempo.

3. Preparar y predecir (0–8)

Recuperar E1 y trabajar en una copia autorizada. Tener a mano `contrato_pipeline.md`, `diccionario_y_reglas.md`, `ventas_original.csv`, `politica_margen_v1.md`, `politica_margen_v2.md` y `matriz_8_pruebas.csv`. El script `pipeline.py` y las carpetas de resultados se crearán en esta copia, separados de las entradas.

Responsabilidades. El agente programador genera, explica y ejecuta cuando tiene herramientas. El revisor contrasta código y contrato; puede ser otro rol o conversación, lo que no garantiza independencia. El estudiante anticipa resultados, revisa el plan y conserva evidencia.

Ruta principal: Work/Codex con archivos y ejecución disponibles. Alternativa: análisis de archivos en ChatGPT con ejecución de Python y descarga de productos. Si solo se puede generar código, el docente ejecuta y el estudiante registra **ejecución docente observada**. La referencia `pipeline_referencia.py` es una contingencia docente; primero se genera una solución propia.

Actividad por chat: predecir antes de ejecutar

Responder con tres decisiones: ¿qué debe ocurrir con una repetición exacta, una cantidad negativa y un precio vacío? Después, indicar qué cambiaría si el encabezado fuera distinto. El docente recoge dos respuestas y las relaciona con excluir o detener.

Dato de contexto, no regla universal: el archivo maestro tiene 16 registros de datos. El programa debe aceptar archivos pequeños con el mismo esquema; una entrada de prueba de 2 filas es válida. Nunca programar `si filas != 16: detener`.

4. Generar y leer decisiones (8–23)

Prompt 1: generar desde el contrato

Actúa como agente programador de Comercial Horizonte. Usa solo los archivos del caso autorizados. Genera pipeline.py con biblioteca estándar de Python, organizado en leer, validar, calcular y guardar. Conserva la interfaz del contrato: –entrada, –politicas, –fecha-ejecucion y –salida. Explica cada bloque en lenguaje simple y presenta el plan y los destinos antes de ejecutar.
Respeta encabezado exacto y origen de registros. Acepta cualquier cantidad de registros con ese esquema: 16 es el tamaño del maestro, no una condición general. Valida ID, fecha real, categoría, cantidad entera positiva, precio positivo, costo no negativo y números finitos. Excluye duplicados exactos posteriores y filas inválidas, registrando motivos y valores originales. Detén ante esquema incompatible, campos inesperados, ID conflictivo o política no determinable. No imputes, no uses valor absoluto y no excluyas una pérdida válida.
Usa Decimal desde el texto, suma antes de dividir y compara el porcentaje antes de redondear para mostrar. Selecciona una política única por la fecha simulada de ejecución, no por la venta. Con cero filas válidas o ventas cero, porcentaje null y estado NO EVALUABLE. Produce ventas_validas.csv, reporte_calidad.json y resumen.json; también JSON en consola. Preserva original y resultados anteriores. Si no tienes ejecución, dilo; no inventes salidas.

Lectura guiada: pedir al agente que señale en su script dónde detecta el encabezado, dónde conserva la primera aparición, dónde divide después de sumar y dónde guarda. No es necesario comprender cada símbolo para relacionar una condición con una regla.

Los siguientes son **fragmentos parciales de lectura**, no un pipeline para copiar y ejecutar. Suponen variables ya preparadas y omiten los controles restantes, la lectura y la selección de política. La variable `destino` representa una carpeta de salida nueva ya creada. El modo `x` evita reemplazar un archivo existente.

```
if encabezado != encabezado_esperado:
    raise ValueError("Encabezado incompatible: revisar entrada")

motivos = []
if fila["precio_unitario"].strip() == "":
    motivos.append("precio vacio")
# Revisar tambien las otras reglas antes de decidir.

ventas_total = sum(ventas_validas, Decimal("0"))
margen_total = sum(margenes_validos, Decimal("0"))
if ventas_total == 0:
    porcentaje = None
else:
    porcentaje = Decimal("100") * margen_total / ventas_total

# Para JSON, conservar Decimal como texto; None se guarda como null.
resumen = {"ventas": str(ventas_total),
           "porcentaje": None if porcentaje is None else str(porcentaje)}
with open(destino / "resumen.json", "x", encoding="utf-8") as archivo:
    json.dump(resumen, archivo, ensure_ascii=False, indent=2)
```

Una fila puede tener varios motivos: el programa completo recoge todos los detectados y excluye esa fila una sola vez. **Por chat:** ¿qué línea decide detener? ¿Cuál registra un motivo de exclusión? ¿Qué diferencia hay entre `None` y cero? ¿Por qué primero aparece `sum` y después la división? Señalar estas decisiones en el script generado, que sí debe tener todos los controles.

5. Ejecutar el caso base y revisar P1–P6 (23–38)

Prompt 2: ejecutar y mostrar evidencia

Revisa el script contra el contrato. Ejecuta con fecha SIMULADA 2026-10-18 y ambas políticas disponibles, dentro de la carpeta autorizada, con salida nueva en resultados_v1. Muestra el comando real, la versión de Python observada y los archivos producidos. Comprueba el original antes y después por una comparación o huella realmente obtenida. Abre los tres productos y relaciona cada exclusión con su origen. Completa P1–P6 con esperado, observado y evidencia; varias pruebas pueden citar la misma ejecución. Distingue estado de la prueba y modalidad individual o docente observada. Si nadie ejecutó, registra pendiente.

Desde la carpeta con script, entradas y políticas, la interfaz prevista es:

```
python pipeline.py --entrada ventas_original.csv --politicas . --fecha-ejecucion
↪  2026-10-18 --salida resultados_v1
```

El punto indica la carpeta actual. Puede corresponder `python3` según el intérprete disponible; el agente lo verifica. No memorizar comandos ni instalar dependencias para esta práctica.

Prueba	Qué comprobamos en la ejecución base
P1	Se leen los 16 registros del maestro y concilian leídos, válidos y excluidos.
P2	V005 se computa una vez; la repetición exacta posterior queda registrada.
P3	La cantidad negativa se excluye sin convertirla con valor absoluto.
P4	El precio vacío se excluye sin inventar un valor.
P5	Los agregados por categoría y total se contrastan con el docente.
P6	El porcentaje global es razón de sumas, no promedio de porcentajes.

Contraste manual breve. Con V001 y V002: 12 unidades; ventas 1450; costos 990; margen 460 UM. Por tanto,

$$100\,\frac{460}{1450} = 31{,}724137\ldots\,\% \quad \longrightarrow \quad 31{,}72\,\%\ \text{mostrado.}$$

Promediar 40 % y 30 % daría 35 %: es incorrecto porque las ventas de ambos registros son diferentes. Esta referencia es un cálculo, no evidencia de haber ejecutado el script.

Pregunta por chat: si el resumen parece razonable, ¿qué archivo abrirías para comprobar que V005 participa una sola vez? Identificar también origen 14 como duplicado, 15 como cantidad inválida y 16 como precio vacío.

6. P7: políticas, igualdad y redondeo (38–53)

Conservar datos y script. Ejecutar también con fecha simulada `2026-10-19` y destino nuevo. La política v1 cubre hasta `2026-10-18` inclusive: alerta si el porcentaje global sin redondear es menor que 25. La v2 rige desde `2026-10-19`: alerta si es menor que 30. La igualdad es **SIN ALERTA**. No se inventan alertas por categoría.

Prompt 3: comprobar qué cambia y qué permanece

Completa P7 en la matriz existente. Ejecuta el maestro con 2026-10-18 y 2026-10-19, conservando script, datos y políticas. Usa carpetas de salida distintas. Compara importes, exclusiones, política elegida y estado. Después ejecuta los dos fixtures de frontera de esta guía con 2026-10-19. Muestra el porcentaje usado para comparar, el mostrado y el estado. No cambies el original; no marques una prueba aprobada sin evidencia.

Fixture 1: guardar aparte como `frontera_30.csv`.

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
T030,2026-10-14,Oficina,1,100,70
```

Esperado con v2: ventas 100, margen 30, porcentaje 30 %, **SIN ALERTA**. La regla dice menor que, no menor o igual.

Fixture 2: guardar aparte como `redondeo_29999.csv`.

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
T029,2026-10-14,Oficina,1,100,70.001
```

Esperado con v2: margen 29,999; porcentaje sin redondear 29,999 %; mostrado 30,00 %; **ALERTA**. Comparar el valor mostrado ocultaría que aún es menor que 30. En el CSV se usa punto decimal.

Por chat, antes de ejecutar: ambos casos pueden mostrar 30,00 %. ¿Deben tener el mismo estado? Responder con la regla que justifica la decisión. Estos fixtures concretan P7; no agregan filas a la matriz ni modifican el maestro.

7. P8 y una detención visible (53–63)

Fixture 3: guardar como `sin_validas.csv`. Contiene únicamente los registros originales 15 y 16, sin cambiarlos:

```
id,fecha,categoria,cantidad,precio_unitario,costo_unitario
V014,2026-10-13,Hogar,-2,75,50
V015,2026-10-14,Oficina,7,,11
```

En esta entrada la numeración local es 1 y 2. Anotar la correspondencia: local 1 ↔ original 15 y local 2 ↔ original 16. Ejecutar P8 con fecha simulada `2026-10-18` y la política v1 válida para esa fecha. Se esperan dos leídas, cero válidas y dos excluidas. Las sumas del conjunto vacío son cero; el porcentaje es `null` y el estado **NO EVALUABLE**. El proceso pudo completar la validación: esto es diferente de un contrato incompatible.

D1: demostración docente de detención. Guardar aparte el cuarto fixture como `esquema_incompatible.csv`. Solo cambia el nombre de la columna de precio:

```
id,fecha,categoria,cantidad,precio,costo_unitario
V001,2026-10-05,Oficina,10,25,15
```

El docente ejecuta con fecha simulada `2026-10-18`, ambas políticas disponibles y destino nuevo `resultados_D1`. Se espera detención con un mensaje comprensible y señal de fallo, sin tres productos nuevos presentados como ejecución exitosa. No renombrar automáticamente la columna. Si quedan archivos de una ejecución anterior, no atribuirlos a la actual.

Actividad por chat: excluir o detener

Completar: «En P8 puedo terminar porque . . . ; en D1 debo detener porque . . . ». Registrar el mensaje real y quién ejecutó D1 en la bitácora. D1 es una demostración de una regla existente: no es P9 ni cambia la rúbrica.

8. R1: repetir la misma versión (63–73)

Volver al maestro y a la fecha `2026-10-18`. R1 se hace con la misma versión base utilizada en `resultados_v1`. **No pedir al agente que regenere el script.** Reutilizar exactamente el mismo archivo, datos, políticas, fecha simulada e intérprete observado; cambiar solo el destino a `resultados_repeticion`. Si la ejecución base falló, registrar el diagnóstico y no aceptar R1 como demostración de éxito.

Prompt 4: repetir y comparar

Reejecuta la misma versión base de pipeline.py, sin regenerarla ni modificarla, con ventas_original.csv, las mismas políticas, fecha simulada 2026-10-18 y el mismo intérprete. Guarda en resultados_repeticion. Compara con resultados_v1 los campos de negocio: filas aceptadas y excluidas, orígenes, motivos, conteos, importes, porcentaje, política y estado. Identifica por separado metadatos que pueden variar, como hora real, identificador de ejecución o ruta de salida. Registra comandos y diferencias reales en la bitácora. No afirmes igualdad sin comparar; si la base falló, conserva el diagnóstico sin aprobar R1.

Qué esperamos: mismos resultados de negocio bajo las mismas condiciones. Los archivos no necesitan ser idénticos byte a byte si contienen metadatos distintos. Repetir con éxito muestra estabilidad

para esta entrada; no demuestra que el cálculo sea correcto para cualquier caso, por eso conservamos P1–P8.

Por chat: ¿regenerar el código con el mismo prompt cuenta como repetir la misma versión? Explicar por qué no. R1 se anota en la bitácora existente; no es una novena prueba obligatoria ni un nuevo entregable.

9. Corrección focalizada y regresión (73–83)

Prompt 5: reparar un fallo comprobado

Usa este fallo documentado: [entrada, esperado, observado y evidencia]. Explica la causa y corrige solo la implementación. Conserva la versión anterior y guarda una nueva. No cambies reglas, original ni esperado para ocultar el problema. Repite ahora la prueba fallida y conserva comando y salida. Ejecuta después P1–P8 como regresión; si no alcanza el tiempo, deja las restantes pendientes para las 2 horas autónomas de la semana. Si ninguna prueba falló, no inventes una corrección: explica qué evidencia permite una aceptación provisional.

Una vez modificado el script, los resultados anteriores ya no bastan para aprobar la nueva versión. La matriz debe indicar a qué versión pertenece cada ejecución. Un segundo rol que responde «todo correcto» no reemplaza el contraste.

10. Tres productos y cierre E2 (83–90)

Producto	Abrir y comprobar
`ventas_validas.csv`	Registros aceptados, origen e importes; original intacto.
`reporte_calidad.json`	Conteos conciliados, valores originales y motivos de exclusión.
`resumen.json`	Categorías y total, razón de sumas, porcentaje mostrado, política y fecha simulada.

E2 amplía E1: conservar script generado y revisado, tres productos, matriz P1–P8 y bitácora acumulativa con prompts, comandos, versión de Python y revisión humana. D1 y R1 se registran en esa misma bitácora. No crear entregas adicionales ni cambiar los criterios de evaluación del curso.

Para cada prueba conservar dos datos: **estado** (aprobado, fallido o pendiente) según la evidencia y **modalidad** (ejecución individual o docente observada). La anotación **observado** de la plantilla indica esta modalidad; no equivale por sí sola a aprobado. Si nadie ejecutó la prueba, sigue pendiente. Una salida de referencia no demuestra ejecución propia.

Salida por chat: tres frases

1. Una regla que comprobé y el archivo que lo demuestra.
2. Qué pasó al repetir la misma versión o al observar D1.
3. Qué queda pendiente y cómo lo completaré.

Trabajo autónomo de la Semana 2: 2 horas en total. Completar pruebas o regresión pendientes, revisar explicación y productos, y ordenar E2. No se agregan dos horas nuevas por esta clase. Si falta ejecución individual, conservar la evidencia docente y declarar el límite. En la Clase 5, el analista reutilizará estos productos; su explicación tendrá una base rastreable y comprobable.